WordForge

1. Project Overview

Provide a brief overview of your project, explaining the game's concept, mechanics, and functionality.

This project is a word-puzzle battle game inspired by Bookworm Adventures, in which players form words from a grid of letter tiles to attack enemies. The player connects adjacent letters to create valid words, which deal damage based on word length and tile effects. The game features an infinite-level system in which enemy difficulty increases via mathematical scaling. Special colored tiles provide additional effects such as bonus damage, healing, or score multipliers to add strategic gameplay.

2. Project Review

Study and then present a relevant existing project, outlining the specific improvements you plan to make. Highlight the key areas where your project enhances or builds upon the original.

An existing project relevant to this game is Bookworm Adventures, a word puzzle RPG where players create words from letter tiles to attack enemies and progress through predefined stages. The game also includes various passive skills and special bonuses tied to specific word types.

The proposed project improves on this concept by introducing an infinite-level system where enemy difficulty increases dynamically using mathematical calculations instead of fixed stages. Additionally, the design simplifies gameplay by removing passive skill mechanics while still keeping special colored tiles that provide strategic effects. This creates a more scalable and focused gameplay experience while maintaining the core word-puzzle combat mechanic.

3. Programming Development

3.1 Game Concept

Describe the game in detail, including its mechanics and objectives. Highlight the key features or selling points of the game.

The game is a word-puzzle battle game in which players create words by connecting adjacent letters on a grid of tiles. Each valid word deals damage to an enemy based on the word length and special tile effects. The game features an infinite-level system in which enemy difficulty increases through mathematical scaling. Special colored tiles provide additional effects such as extra damage, healing, or score bonuses, adding strategic elements to the gameplay.

3.2 Object-Oriented Programming Implementation

List and describe at least five classes you will implement in your game. For each class, briefly explain its role, key attributes, and methods. You can also include a UML diagram for better understanding. (Later, this UML will be required as part of the full proposal.)

Core Game

- Game - Main game controller; initializes pygame, manages the game loop, scenes, player, enemy, and save/load logic.
- SceneManager - Manages scene switching between menu, gameplay, and stats.

Scenes

- BaseScene - Abstract base for all scenes; handles background layers and button rendering.
- MenuScene - Main menu with Start, Stats, and Quit buttons.
- GameplayScene - Core gameplay screen; manages the board, attack flow, turn logic, HUD, and overlays.
- StatsScene - In-game stats screen that launches the stats window and offers reset/delete actions.

Entities

- Player - Player entity; handles HP, animations, attack resolution, abilities, and damage.
- Enemy - Enemy entity; handles HP, attack sequences, abilities, and animations.

Board & Tiles

- Board - 4*4 letter grid; manages tile selection, word validation, attacks, gravity, and board regeneration.
- Tile - Individual letter tile; handles rendering, animation, selection state, and ability color.

UI Components

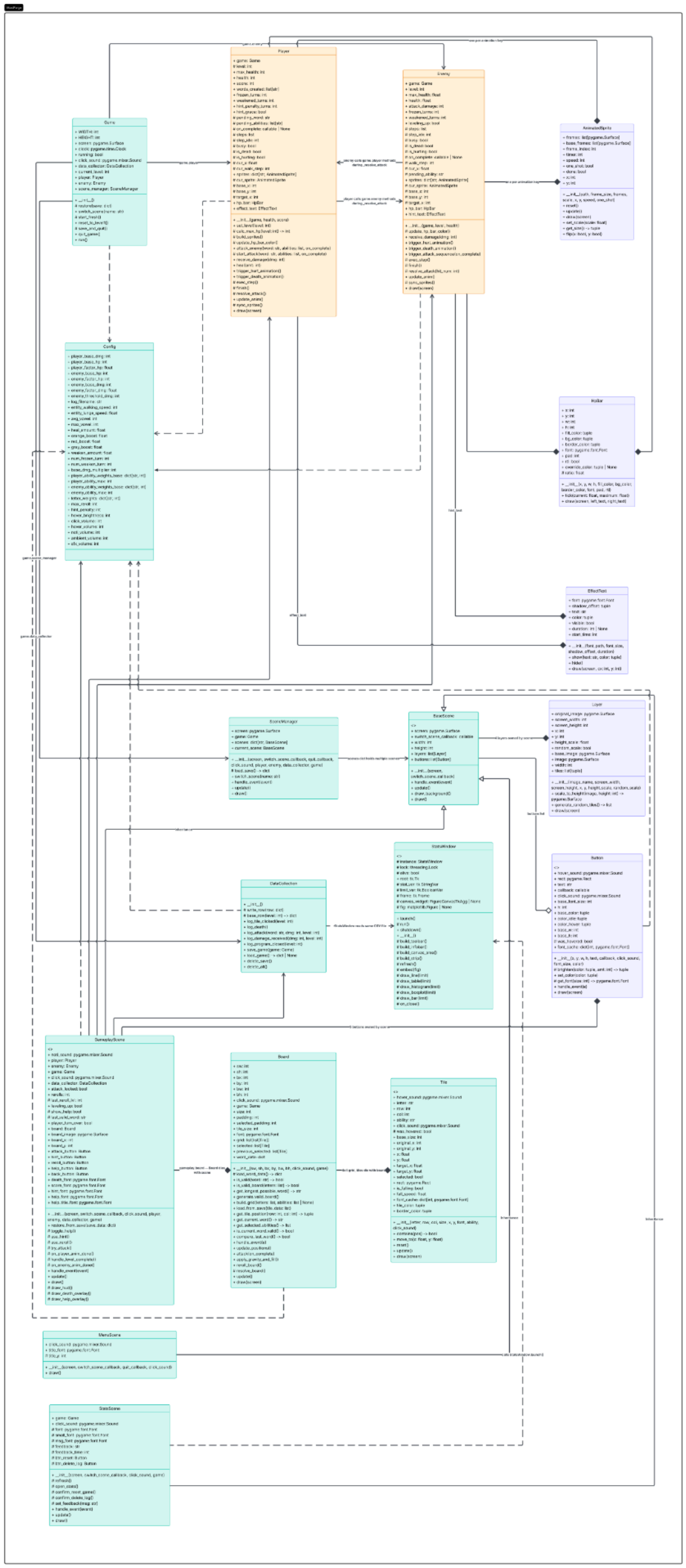
- Button - Clickable button with hover/click effects and sound.
- HpBar - Animated health bar with status effect color overrides.
- EffectText - Timed floating text label for ability feedback.
- AnimatedSprite - Sprite sheet animator with scaling, flipping, and one-shot support.
- Layer - Parallax background layer with optional random tile scaling.

Data & Stats

- Config - Central configuration constants (damage formulas, ability weights, sound names, etc.).
- DataCollection - CSV logging and JSON save/load management.
- StatsWindow - Tkinter-based stats dashboard with matplotlib charts (line, table, histogram, boxplot, bar).

UML:

https://github.com/CYRTRUS/final_project_y1_ske23/blob/main/WordForge_UML_Class_Diagram.pdf


Player
Enemy
Game
AnimatedSprite
Config
HpBar
EffectText
SceneManager
BaseScene
Layer
StatsWindow
Button
DataCollection
GameplayScene
Board
Tile
MenuScene
StatsScene

3.3 Algorithms Involved

Mention the algorithms used in the game. If your game incorporates techniques such as pathfinding, sorting, AI behavior, rule-based logic, or event-driven mechanics, describe them here.

1. Word Validation Algorithm

The game uses a dictionary-based lookup to verify player-formed words. At startup, Board._load_word_data() pre-indexes the entire word list into a nested dictionary keyed by word length and first letter - data[length][first_letter]. When validating, Board.is_valid() looks up only the matching length and first-letter bucket, reducing comparisons significantly instead of scanning the full dictionary linearly.

2. Board Generation Validation Algorithm

Before finalizing any board state, Board.is_valid_board() uses a Counter-based subset check to confirm that at least one 3-letter word from the dictionary can be formed from the available letters. If no valid word exists, Board.generate_valid_board() regenerates the entire letter set and retries in a loop until the condition is satisfied. The same check runs after gravity refills (Board._resolve_board()) to ensure playability is always maintained.

3. Board Tile Update Algorithm

After a valid word attack, Board.apply_gravity_and_fill() processes each column independently from bottom to top, compacting non-empty tiles downward and tracking empty slots. New letters are generated to fill those slots, with vowel count actively managed against Config.max_vowel to preserve board solvability. New tiles are spawned above the visible board area with is_falling = True and smoothly interpolated to their target position each frame inside Tile.update() using a fixed lerp factor of 0.15 horizontally and 0.2 vertically.

4. Level Scaling Algorithm

Enemy stats scale using explicit linear formulas defined in Config:

- max_health = enemy_base_hp + enemy_factor_hp * (level - 1)
- attack_damage is randomized within a threshold band: base_dmg + factor_dmg * (level - 1) * (1 ± threshold/100)

Player max HP follows the same pattern: player_base_hp + player_factor_hp * (level - 1). Player attack damage scales as len(word) * ceil(level^0.55) + player_base_dmg, using a sub-linear power curve so damage growth slows at higher levels. Ability roll weights for both player and enemy also increase with level, capped by player_ability_max and enemy_ability_max in Config.

5. Rule-Based Tile Effect Algorithm

During Player._resolve_attack(), each tile ability in the selected word is counted and processed in a fixed priority order: green -> blue -> purple -> orange -> red -> gray. Green triggers healing, blue applies freeze turns to the enemy, purple applies weaken turns, and orange/red/gray stack as exponential damage multipliers (boost^count). An additional letter-pattern bonus applies a 1.05* multiplier per occurrence of x, y, z, or words ending in r, y, es, or a consonant-s. The final damage is ceil(base_dmg * combined_multiplier), with weaken halving the multiplier if active. Enemy abilities follow a symmetric rule-based system in Enemy._resolve_attack(), applying healing, freeze, or weaken to the player depending on the rolled ability.

4. Statistical Data (Prop Stats)

4.1 Data Features

Describe at least five features you will track in the game, which may include player behavior or game-related metrics (e.g., character movement, keystrokes, pixels moved, player score, time played, enemies defeated, accuracy, and completion rate). Each feature should have at least 100 rows of data. (For example, in football, a player's performance can be analyzed by collecting and analyzing statistical data.)

Feature	Why is it useful?	How to get 100 values?	Variable & Class	Display
Tile Clicking (every 20s)	Detects spam vs strategic play	Count clicks every 20s across sessions	DataCollection (click)	Line Graph (Time vs Clicks)

Words Created (by length)	Shows preferred word length	Record words and group by length	DataCollection (word_length)	Table (length, count)
Damage Dealt (overall)	Shows damage distribution	Record all damage events	DataCollection (damage_dealt)	Boxplot (Damage distribution)
Word Length	Shows combat consistency	Record words	DataCollection (word_length)	Histogram (Word length distribution)
Time to Complete Level	Shows level difficulty	Group level times (<1, 1-5, >5 min)	DataCollection (level, timestamp)	Bar Graph (Time category vs Count)

4.2 Data Recording Method

Explain how the game will store statistical data. Will it be saved in a database, CSV file, or another format?

The game stores statistical data in a CSV file using an event-based recording system. Each row represents a single gameplay event with a timestamp. The file is managed by the DataCollection class in data_collection.py, and the path is defined as lib/stats/log_1.csv via Config.log_filename.

CSV columns (header):

- timestamp - time when the event occurs, formatted as YYYYMMDD_HHMMSS
- tile_clicked - "True" if a tile was clicked, otherwise empty
- damage_received - damage taken by the player from the enemy, otherwise empty
- created_word - the word string submitted by the player, otherwise empty
- damage_dealt - damage dealt to the enemy, otherwise empty
- current_level - the level number at the time of the event
- program_closed - "True" if the game was closed during this event, otherwise empty

Each event type is written by a dedicated method. log_tile_clicked() records tile interactions, log_attack() records the submitted word and damage dealt, log_damage_received() records incoming damage, log_death() records player death, and log_program_closed() records when the game exits.

Every row is sparse - only the fields relevant to that event are filled, and the rest are left empty.
The system records raw event data rather than pre-calculated metrics. Statistical features are derived later in stats_data.py by reading and processing the CSV at display time. For example, get_tile_clicked() buckets click events into 20-second intervals, get_word_created() groups words by length, get_damage_dealt() collects damage values for distribution analysis, get_word_length() extracts word lengths for histogram plotting, and get_beat_time() measures time between level changes to categorize completion speed.

4.3 Data Analysis Report

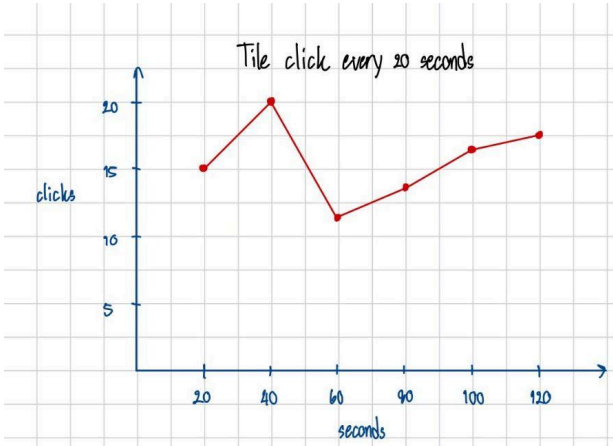
Outline how you will analyze the recorded data. What statistical measures will you use? How will the analysis be presented (e.g., graphs, tables, charts)?

The recorded gameplay data will be analyzed using both statistical measures and data visualization techniques. The analysis will focus on identifying player behavior patterns, performance trends, and game balance across different levels.

Feature	What the Graph Shows / Why	Visualization Method	X-axis	Y-axis
Tile Clicking (every 20s)	Shows click frequency over time. Dense clusters indicate active exploration; sparse periods reflect hesitation.	Line Graph	Time (20-sec intervals)	Number of clicks
Words Created (by length)	Shows the distribution of word lengths used by the player	Table	Word length	Example words per length group
Damage Dealt (overall)	Summarizes all attack damage values showing min, median, max, and outliers. Wide spread reflects	Boxplot	All attacks (single category)	Damage dealt per attack

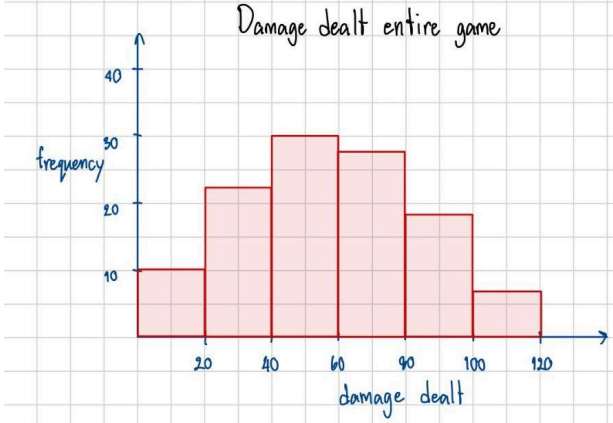
	word length, level scaling, and ability multipliers.			
Word Length Spread	Shows word length frequency across all submissions with a mean line. Skew direction reflects whether the player favors speed or longer, deliberate words.	Histogram	Word length (letters)	Frequency (number of words)
Time to Complete Level	Categorizes completed levels into three time bands. Reflects whether players clear levels quickly or struggle.	Bar Graph	Time category (<1, 1-5, >5 min)	Number of levels

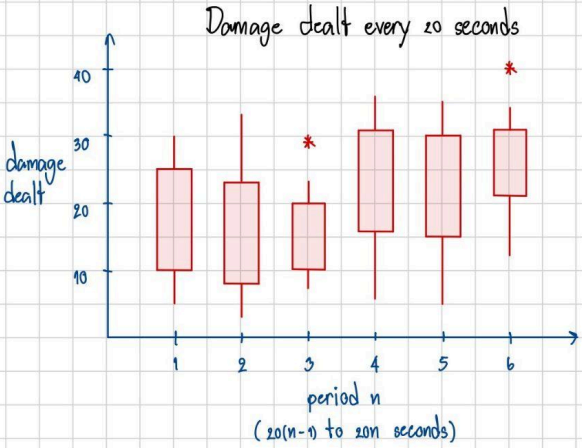
Hand-drawn expected graphs

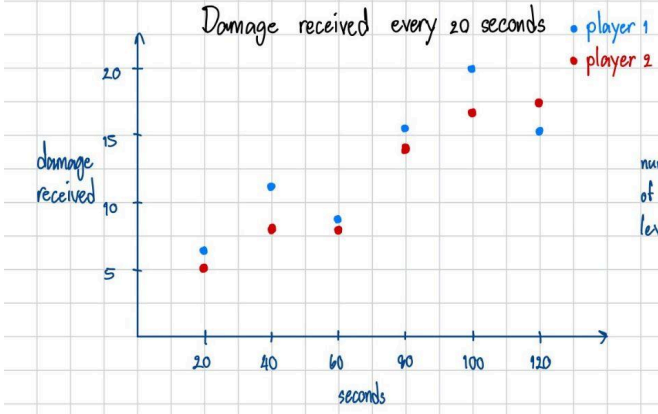


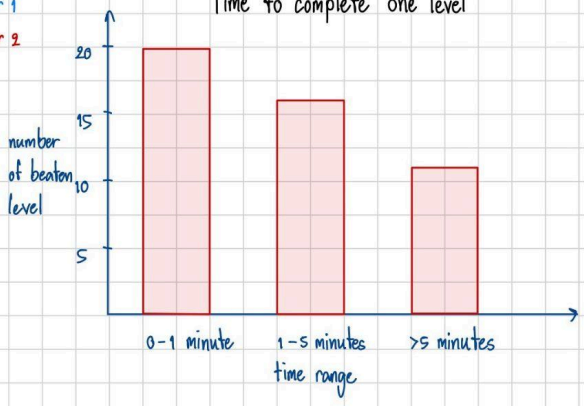
Word created for each length

Length	Word list	Number
3	pie, fox	2
4	same, type, loan	3
5	lotus, grade, fails, loans	4
6	travel, silver	2
7	journey, freedom	2
9	freedoms, mountain, complete	3
10	background, adventure	2









5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Implement basic system (No pygame GUI)
3	10	Create a simple GUI for level 1 gameplay
4	11	Implement special tiles and level scaling
5	12	Implement stats recording
6	13	Let friends test the game and provide feedback.
7	14	Submission week (Draft)

6. Document version

Version: 1.0
Date: 13 March 2026
Editor: Wassawin Swangjaeng

Version: 2.0
Date: 28 March 2026
Editor: Wassawin Swangjaeng

Version: 3.0
Date: 8 May 2026
Editor: Wassawin Swangjaeng

Date	Name	Feedback
21/03	Amornrit	Great detail on the algorithm part. Justify your proposed data features. Provide more details on the Data Analysis Report, how your proposed data features will be analyzed and visualized.